

Part IV

Hands-On GRPO Training

Resources

- torch profiler trace files
- accelerate/torch-run is a distributor training interface
- primeintellect.ai – marketplace for GPU rentals
- unsloth – performance gains

GPU training

GPU training is mostly done for you via accelerator or something else. Logging and checkpoints are important but you do not want to do it on every GPU, so you start adding some guard statements. Distributed training nowadays comes down to setting up a distributed dataloader and distributed optimizer.

What is a rank in distributed training?

What is the training signal?

- Only download the dataset through rank 0, you don't want to do it every time
- Use recipes to install environment, create venv, export envs. GPU libraries are very sensitive. Copy/paste with a lot of waiting.

You can check the rank for checkpoints. GPUs are expensive and difficult to get working.

You need to ssh into these remote instances. The first window that opens will close as soon as you use it.

Training takes a lot of memory compared to generation. The solution is performing gradient accumulation steps. Batching is about the math, not the parallelism. When you batch a bunch of training examples, you're calculating gradients for all of

those providing a training signal. You're averaging out the gradient you going to update based on a larger collection. From a training perspective, there's an ideal batch size since it's good mathematically and creates effective training updates, and its picked based on training dynamics. This ideal batch size may not fit since its too large, which is okay because you can slice a batch and process it in smaller chunks, which is mathematically equivalent to summing together. 8 prompts, 8 sequences to train on, and take the first four, run forward, calculate loss, get gradients, and before running the optimizer, do it again on the other half, then sum those two gradients and give the result to the optimizer. Even though it's slower because we've incurred overhead by doing two separate runs of two forward and backward passes, but the point is we end up getting the batch size we want.

The optimizer is just there to smooth out the gradient updates.

For GRPO, you want to see the rewards increasing over time in addition to the loss reducing over time. We log the average reward as well as the standard deviation (STD) because the STD reflects the diversity in the rewards. Low STD implies the outputs all getting the same reward so the model isn't learning – the training signal will not be very large. You can also calculate the fraction of prompt batches are generating groups where the rewards are identical across all the responses – not teaching the model anything. You want this to be low. It needs to climb for the reward to go up, but the higher it is the less signal you're getting out of expensive batches. How does this happen? If your generated data isn't feeding the model the correct difficulty of problems, you need to balance the difficulty. Generate a bunch of problems, have the base model do it, and calculate the reward.

- Greedy result – most likely next token, or beam search most likely sequence of tokens
- Easy? All questions are correct
- Hard? All questions are wrong
- Medium/Medium-Hard – Some correct

Gradient checkpoints – slows things down.

Rejection sampling. Take training set, generate responses, keep all that are right, and use that as a fine tuning dataset.

Rounding matters significantly. Changing the boundary of a kernel can result in substantial changes.

Best step – greedy decoding, validation run every 20 steps. Then saving checkpoint. Final result or test score is compared against best validation checkpoint, faster than running different checkpoints against the test set. Performance and accuracy bounce around a lot.

Will GRPO penalize the model for the bad responses? Is that the effect of the loss function? Weigh less good ones negatively. You want to optimize that a group has correct and incorrect responses. Otherwise the training signal... if everything fails, the model doesn't really learn very much, whereas if everything is successful, then ????. With the imbalanced dataset, the average reward is good so the model doesn't learn that much.

Different runs can run into different sequence lengths. Imagine really run-on answers that are just incorrect. Truncated, we cut it off since it exceeds. Mask the truncated completions, don't include those in the training.

Left off at 1:18:30